西北大学

第四章：GDAL 入门基础

谭振宇

西北大学城市与环境学院

2026 年 05 月 06 日

引言与简介

空间矢量数据

空间栅格数据

工具集与自动化处理

本章小结

前言

在掌握了 Python 编程语言基础、GIS 基本概念以及空间数据相关知识之后，本章正式进入空间数据处理基础库 **GDAL**（Geospatial Data Abstraction Library）的学习。

> GDAL 是一个用于对空间矢量和栅格数据进行读取、操作和格式转化的开源库，使用 C++语言进行编写，通过一个抽象的矢量和栅格数据模型屏蔽了不同数据底层的格式差异，提供了统一的数据访问接口。

不管是商业软件 ArcGIS 还是开源软件 QGIS，大都使用了 GDAL 作为底层构建库，所以学习和掌握 GDAL 对于空间数据的批处理或自定义数据处理流程十分必要。

学习目标

本章将涵盖以下几个核心目标：

环境与模型

- 掌握 GDAL 开发环境搭建。
- 掌握 GDAL 空间数据模型的基本知识（栅格与矢量）。

数据读取与操作

- 能够使用 GDAL 进行矢量地图的读取。
- 能够使用 GDAL 进行栅格影像的读取。
- 能够对空间数据进行投影转换和简单的批处理。

GDAL 简介

GDAL (Geospatial Data Abstraction Library) 是使用 C/C++ 语言编写的用于读写空间数据的一套跨平台开源库。

目前的 GDAL 库由原来的 **OGR** 和 **GDAL** 项目合并而来：

- **OGR**：主要用于空间要素 **矢量数据**的解析。
- **GDAL**：主要用于空间 **栅格数据**的读写。

数据的空间参考及其投影转换使用开源库 PROJ.4 进行。

统一数据模型

GDAL 对多种多样的空间数据格式进行抽象建立统一的数据模型，并提供统一的 API，使不同格式数据读写具有一致性。

支持的类型

主要提供了对三大类数据的支持：栅格数据模型、矢量数据模型及空间网络数据模型。

GDAL 的演进与发展

在进行脚本开发前需要注意 GDAL 版本的演变。

GDAL 1.x 版本

栅格和矢量数据的读写接口设计是相对分离的：

- `GDAL` 模块用于栅格数据的访问。
- `OGR` 模块用于矢量数据访问。

GDAL 2.x 版本及之后

进行了接口统一，合并了相关对象：

- 栅格：原 `GDALDataset`
- 矢量：原 `OGRDataSource`

新版中均可通过共同的 `GDALDataset` 类进行处理。但旧版 `OGRDataSource` 仍可使用。

多语言支持：虽然使用 C++ 编写，但是通过 SWIG 提供了 Python, Java, C# 等语言接口。Python 可以通过额外的方法将图像数据转为 NumPy 的 `ndarray` 加以融合处理。

空间数据处理环境搭建

对于底层使用 C/C++语言开发的 Python 库，推荐使用 **Conda 工具** 构建虚拟环境进行包的安装。

搭建过程 (Windows/Linux)

首先使用 `conda create` 命令创建一个专门用于空间数据处理的虚拟环境：

```
# -n参数指定环境名称 osgeo，这里指定 Python 3.7+ 版本
conda create -n osgeo python=3.7

# 切换当前环境到刚新建的虚拟环境 osgeo
conda activate osgeo

# 使用conda install命令，从 conda-forge 仓库进行 GDAL 的安装
conda install -c conda-forge gdal
```

至此我们将会在 `osgeo` 的虚拟环境下进行书中的全部操作。

引言与简介

空间矢量数据

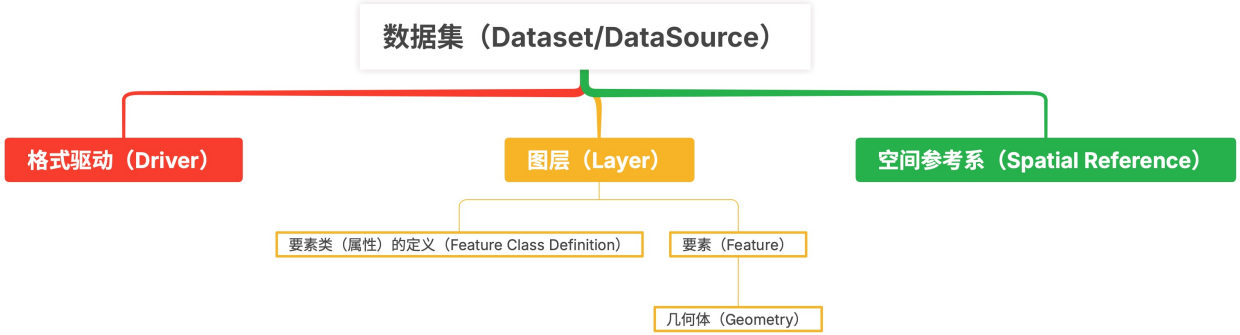
空间栅格数据

工具集与自动化处理

本章小结

矢量数据组织

矢量数据模型主要根据开放地理数据联盟提出的 **简单要素规范（OGC SFS）** 进行设计，并使用 C++ 语言对规范中的概念使用类方法进行实现。以常见的 **Shapefile** 为例：



- **Dataset**：代表数据集 (如一个 shp 或 gdb)。
- **Layer**：数据集包含图层。
- **Feature**：图层中的每个要素实体。
- **Geometry**：要素关联的几何体。
- **Field**：每个要素上的属性值。

矢量数据核心对象 (一)

西北大学

数据集（DataSource/Dataset）

GDAL 2 之后，`OGRDataSource` 继承自 `GDALDataset` 对象，用于表示一个矢量数据或矢量数据库，里面包含一个或多个 `OGRLayer` 对象。

图层（Layer）对象

`OGRLayer` 类用来描述数据集 `DataSource` 中的一个图层中包含的所有空间要素的集合。

空间要素（Feature）对象

`OGRFeature` 类对应一个空间实体，由一个几何体对象和一组属性组成。

属性定义（FeatureDefn）

提供每个图层的统一属性字段定义格式，GDAL 通过 `OGRFeatureDefn` 类封装要素的属性表定义元数据。

矢量数据核心对象 (二)

西北大学

几何体（Geometry）对象

一个要素对应一个几何对象，`OGRGeometry` 类根据 OGC 简单要素规范实现，定义了几何体模型（如 Point, LineString 等）和相关操作，支持常用二进制格式和文本在内的转换。

空间参考（SpatialReference）

`OGRSpatialReference` 对象封装了数据使用的空间投影参考信息。可以通过 DataSource 或 Layer 获取。也提供了 OGC WKT 与 PROJ4 的文本互转操作。

在 Python 获取相关的对象时，往往通过函数链式的方法如 `layer.GetFeature()` 或者 `feature.GetGeometryRef()` 等实现上下游的获取。

矢量数据读取

如果完全自主解析 Shapefile，需要分别读取 `.shp`（地理对象），和 `.dbf`（属性表），非常复杂。而在 GDAL 里面我们可以通过两行代码打开它：

`gdal.OpenEx()` 与 `ogr.Open()` 的比较

- 在新版中，`gdal.OpenEx()` 返回的是通用的 `gdal.Dataset`，能够同时读取或分析栅格数据和矢量数据。
- 旧版和部分模块中保留的 `ogr.Open()` 只能返回矢量数据 `ogr.DataSource`。
- 对于基于矢量处理的需求二者后续操作逻辑类似。

由于 Python 是动态强类型语言，我们在后续代码中加入了变量的 **模式标注 (Annotation)**，如 `ds: gdal.Dataset = x`，方便追踪与学习内部对应的数据类型。

示例：使用 gdal 读取 Shapefile (1/2)

西北大学

```python
from osgeo import gdal, gdalconst, ogr, osr

fn = r"C:\Data\chn_admbnda_adm1_ocha_2020.shp"
# gdal.OpenEx 参数中 OF_VECTOR 指定打开矢量格式
ds: gdal.Dataset = gdal.OpenEx(fn, gdalconst.OF_VECTOR)

# 从 Dataset 读取具体的图层
lyr: ogr.Layer = ds.GetLayerByName('chn_admbnda_adm1_ocha_2020')
srs: osr.SpatialReference = lyr.GetSpatialRef()
print(f'空间参考信息: {srs}')

# 读取所有的属性表字段定义
defn: ogr.FeatureDefn = lyr.GetLayerDefn()
for i in range(defn.GetFieldCount()):
    fd: ogr.FieldDefn = defn.GetFieldDefn(i)
    print(f'字段名称: {fd.GetName()}\t类型: {fd.GetTypeName()}')
```

示例：使用 gdal 读取 Shapefile (2/2)

西北大学

```python
# 根据索引打开对应具体的空间要素 Feature
feat: ogr.Feature = lyr.GetFeature(0)

# 获取 Feature 保存的几何坐标信息
geom: ogr.Geometry = feat.GetGeometryRef()
print(geom.ExportToWkt())  # 导出WKT文本显示

# 获取 Feature 对应的各项属性表的值
print('FID为0的要素的属性值: ')
for i in range(feat.GetFieldCount()):
    print(feat.GetField(i))

del ds # 释放占用
```

通过类似的方法一样也可以使用 `ogr.Open(fn)` 进行等价替代执行，不同区别在于省略部分读取模式选择，并返回特定的 DataSource 实例。

矢量数据创建与保存的步骤

创建一条完整的 Shapefile 数据需要依照一定的顺序装配各个对象层级：

1. 创建结构框架

- 1. 使用 `ogr.Driver` 的 `CreateDataSource()` 创建矢量数据集文件。
- 2. 使用 `DataSource` 的 `CreateLayer()` 创建图层。
- 3. 使用 `ogr.FieldDefn()` 定义 Shapefile 文件结构的属性表字段并应用给图层。

2. 装配并写入要素

- 4. 创建 `ogr.Feature` 空白对象。设置每个字段值，并利用 `SetGeometry()` 定义几何坐标。
- 5. 使用 `Layer.CreateFeature()` 将组合好的要素附加到图层里。
- 6. 循环重复操作填充全部数据。

示例：GeoJSON 转为 Shapefile

西北大学

```python
from osgeo import ogr, osr
import json, os
os.environ['SHAPE_ENCODING'] = 'GBK'

# 一：读取 json 内容，创建 shp 和基础坐标系参数
with open(r"C:\Data\china.json", encoding='UTF-8') as f:
    china = json.load(f)
driver = ogr.GetDriverByName('ESRI Shapefile')
ds = driver.CreateDataSource(r"china.shp")

srs = osr.SpatialReference()
srs.ImportFromEPSG(4326) # 构建 WGS 84

# 二：创建图层与列
layer = ds.CreateLayer('province', srs, ogr.wkbPolygon)
fname = ogr.FieldDefn('Name', ogr.OFTString)
fname.SetWidth(24)
layer.CreateField(fname)
layer.CreateField(ogr.FieldDefn('CenterX', ogr.OFTReal))
...
```

示例：GeoJSON 转为 Shapefile (续)

西北大学

```python
# 三：通过循环，逐步组装每个对象并塞入文件
for f in china['features']:
    # 新建Feature
    feature = ogr.Feature(layer.GetLayerDefn())

    # 赋予属性并设置坐标值（可直接解析 geojson 字典里的几何部分）
    feature.SetField('Name', f['properties']['name'])
    feature.SetField('CenterX', f['properties']['cp'][0])
    feature.SetField('CenterY', f['properties']['cp'][1])

    polygon = ogr.CreateGeometryFromJson(str(f['geometry']))
    feature.SetGeometry(polygon)

    # 创建并持久化
    layer.CreateFeature(feature)
    del feature

ds.FlushCache()
del ds
```

矢量数据直接格式转换

虽然通过前面的遍历获取和保存可以在不同的扩展名和格式中流转，但是这太过于冗长：

GDAL 提供了一个极简的包装库函数 `gdal.VectorTranslate()` 以一行式的操作模拟其底层的 `ogr2ogr` 工具对整个数据库进行完整的等价克隆转换。

```python
import os
from osgeo import gdal, ogr

ifn = r"china.shp"
ofn_kml = r"china.kml"
ofn_gpkg = r"china.gpkg"

# 将 Shapefile 转换为 KML 或 GeoPackage
gdal.VectorTranslate(ofn_kml, ifn, format='KML')
gdal.VectorTranslate(ofn_gpkg, ifn, format='GPKG')
```

矢量数据投影转换

我国省区通常使用阿尔伯斯割圆锥投影显示，我们需要改变 WGS84 的原始坐标结构去产生变形展示。投影转换主要有两种形式：

方式 A：使用封装命令函数

利用 `gdal.VectorTranslate/gdalwarp` 或底层的 `ogr2ogr` 命令行工具。直接提供源文件，目标文件，并在参数里写入需要投影的语法（PROJ4 或 EPSG）即可，安全快捷。

方式 B：基于 API 逐项转码

需要对新建文件的每个要素几何体 `Geometry` 调用基于 `osr.CoordinateTransformation` 对象重载的 `.Transform()` 方法，逐步计算出新的投影再录入到图层里。

代码演示：矢量重投影 (封装命令模式)

基于 gdal.VectorTranslate() 可以轻易附加强制重投影逻辑，这种模式无需我们重新从零建立循环和字段格式。

```python
from osgeo import gdal, ogr

ifn = r"china.shp"
ofn = r"china_reprojected.shp"

# 输出数据投影定义（PROJ4文本模式）
srs_def = """+proj=aea +lat_1=25 +lat_2=47 +lat_0=30 +lon_0=105 +x_0=0 +y_0=0
+ellps=WGS84 +datum=WGS84 +units=m +no_defs """

# 调用带目标投影（dstSRS）和强制投影（reproject）的API进行转化
gdal.VectorTranslate(ofn, ifn, dstSRS=srs_def, reproject=True)

# 打开查看是否成功替换:
ds = ogr.Open(ofn)
print(ds.GetLayer(0).GetSpatialRef())
del ds
```

代码演示：矢量重投影 (底层 API)


```python
# 输出数据投影定义
srs_def = "+proj=aea +lat_1=25 +lat_2=47 ..."
dst_srs = osr.SpatialReference()
dst_srs.ImportFromProj4(srs_def)

# 关键：创建基于两套坐标系的映射器
ctx = osr.CoordinateTransformation(src_srs, dst_srs)

... # 此处略过源图层和新图层的创建复制操作

# 循环源文件几何体
src_feat = src_layer.GetNextFeature()
while src_feat:
    geometry = src_feat.GetGeometryRef()

    # 执行坐标转换，覆盖原来的值：
    geometry.Transform(ctx)

    dst_feat = ogr.Feature(layer_def)
    dst_feat.SetGeometry(geometry)

    # 复制属性和提交图层保存的操作
    ...
```

相比命令行模式，底层 API 允许你对几何体的变化做出更加细粒度的逻辑控制和干越。

引言与简介

空间矢量数据

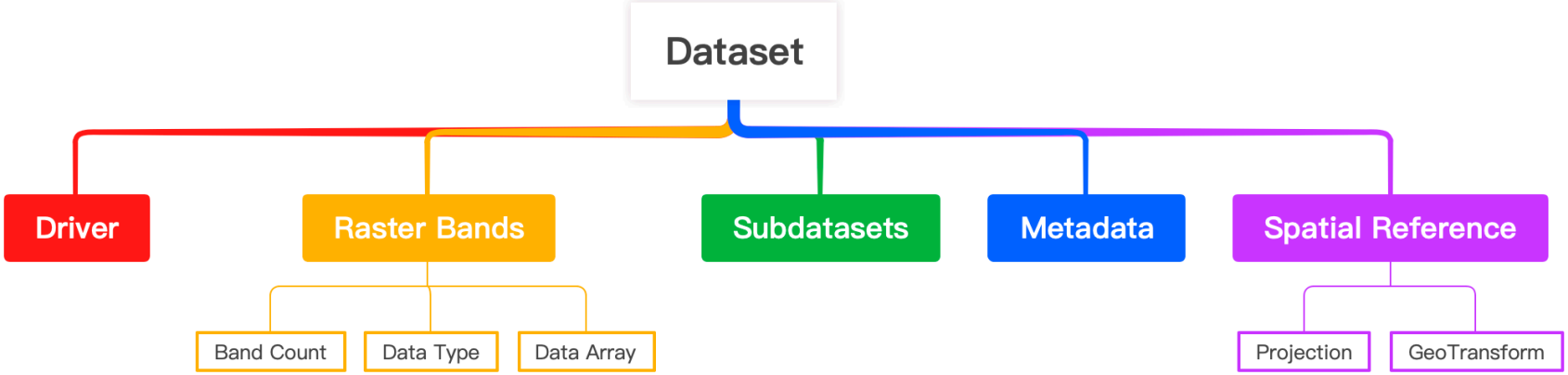
空间栅格数据

工具集与自动化处理

本章小结

栅格数据组织

一幅遥感影像、DEM 高程模型、在 GDAL 中都会被抽象认为一个 `GDALDataset` 类别实例。其中不仅仅装载了数值，还具备有将像素位置与地球真实定位的信息。



栅格数据核心组件

西北大学

空间参考系统（SRS） & 地理防射变换

- SRS 通常用 OGC WKT 表示其投影或地理坐标系统。
- **Affine Transform（仿射变换）** 记录了遥感影像图上坐标 (Col, Row) 和地理坐标 (X, Y) 的映射六个参数关联。

控制点 (GCP) & 元数据 (Metadata)

- 当缺乏直接定位信息时，影像使用地面控制点通过多项式记录变形。
- 属性值包括采集时间记录，来源提供方的标注信息等。

栅格波段（Band）

使用 `GDALRasterBand` 类表示，真正用于储蓄影像值的多矩阵对象（如 Landsat 一般有 11 个波段深度）。

颜色表（Color Table）

对于一些灰度或者分类影像指导 GUI 色块渲染配置表。

深刻理解：地理仿射变换

仿射变换能够使用六项核心数组（GeoTransform），按照指定的伸缩、平移乃至旋转公式解决如何将左上角的 `(x=0, y=0)` 阵列映射到真实的经纬度或米单位坐标：

$$X_{\text{geo}} = \text{GT}(0) + X_{\text{pixel}} \cdot \text{GT}(1) + Y_{\text{line}} \cdot \text{GT}(2)$$
$$Y_{\text{geo}} = \text{GT}(3) + X_{\text{pixel}} \cdot \text{GT}(4) + Y_{\text{line}} \cdot \text{GT}(5)$$

对于非旋转规则的极地图像来说：

- `GT(0)` 和 `GT(3)`：分别是图像最左上角像元的横纵坐标。
- `GT(1)` 和 `GT(5)`：分别是代表图像 X 轴分辨率，Y 轴的分辨率长度（常为像素值或负号等长）。
- `GT(2)` 和 `GT(4)`：常为 0，代表图无偏转。

栅格数据读取与解剖

```python
from osgeo import gdal

# 直接读取数据集
ds = gdal.Open(r"wsiearth.tif")

print(f'投影信息: {ds.GetSpatialRef()}')
print(f'栅格波段数: {ds.RasterCount}')
print(f'尺寸: {ds.RasterXSize} x {ds.RasterYSize}')

# 读取元数据
for key, value in ds.GetMetadata_Dict().items():
    print(f'{key} → {value}')

for b in range(ds.RasterCount):
    # 需要注意的是波段是从 1 开始遍历而非 0
    band = ds.GetRasterBand(b + 1)
    print(f'类型: {gdal.GetDataTypeName(band.DataType)}')
    print(f'NoData掩码极值: {band.GetNoDataValue()}')
    print(f'统计: {band.ComputeRasterMinMax()}')

del ds
```

桥梁：转换影像为 NumPy 多维数组

栅格本质上就是数据网格阵列，我们完全可以将它们剥离 GDAL 的逻辑提取为纯粹的 Python 多维数值分析数组，然后再借助于 `scikit-learn` 之类的进行复杂处理。

方式 A：针对整个数据集层面加载

```
# 会返回（波段数，行宽，列数）
# 的 3D ndarray 数组
img_array = ds.ReadAsArray()
```

方式 B：基于某个单一波段处理加载

```
band = ds.GetRasterBand(1)
# 会返回二维矩阵
band_array = band.ReadAsArray()
```

在获得数据后，可通过对应的 `.WriteArray(np_array)` 在对应坐标段重新覆写回原本空间文件中去。

GDAL 脚本中的错误与异常机制

在新版本的库中，若果不手动声明使用 Python 异常反馈机制往往会导致程序默默崩溃而不是向编译器抛出具体的红字回溯：

```python
import gdal
import sys

# 必须写在程序前段：允许 GDAL 抛出 Python 的 try/except 异常机制
gdal.UseExceptions()

try:
    ds = gdal.Open('error_example.tif')
except (FileNotFoundError, RuntimeError) as e:
    print('文件打开失败或路径错误！')
    print(e)
    sys.exit(1)
```

栅格数据创建与保存

如何从一段经过分析生成的 Numpy 空白或者改写数据创建一个携带合理坐标位置的 GeoTIFF 文件呢？同样我们有两个思路。

原型克隆策略 (CreateCopy)

假设我们对该图像处理前处理后波段数量，形状长宽比例没有实质发生改变，我们完全可以基于原始文件的底层使用 `driver.CreateCopy(filename, prototype)` 初始化后替换内部像素值。

纯手工注入策略 (Create)

无中生有，从零拼装！使用 `driver.Create()` 声明新的变量画布长宽等。然后利用 `dataset.SetGeoTransform()` 和 `dataset.SetProjection()` 手动写入六参数与坐标信息最后填充写盘。

示例：创建一个手工写盘的函数框架

西北大学

```python
def array2raster(filename, np_array, prototype=None,
                 transform=None, projection=None, nodata=None):
    driver = gdal.GetDriverByName('GTiff')

    # 分情况讨论
    if prototype:
        dataset = driver.CreateCopy(filename, prototype)
    else:
        ysize, xsize = np_array.shape[-2], np_array.shape[-1]
        dataset = driver.Create(filename, xsize, ysize, 1, gdal.GDT_Float32)

        # 手动挂在赋予坐标系和偏移六参数
        dataset.SetGeoTransform(transform)
        dataset.SetProjection(projection)

    # 最后统一操作并向第一个Band内把分析出来的数组全覆盖写入并缓存回盘落到 C 接口上
    dataset.GetRasterBand(1).WriteArray(np_array)
    if nodata: dataset.GetRasterBand(1).SetNoDataValue(nodata)

    dataset.FlushCache()
    del dataset
```

栅格数据格式转换

正如矢量可以利用 VectorTranslate 去无脑跨格式转化，遥感栅格文件也可以利用相似的处理去抹平不同厂商文件结构：

命令行直接封装支持

```
# gdal.Translate() 等同外部指令翻译器
ds = gdal.Open('example.tif')
# 将Tiff导出成erdas 的 img格式
gdal.Translate('example.img',
    ds, format='HFA')
```

基于复制功能迁移

```
src_ds = gdal.Open('example.tif')
# 先寻找需要导出的目标类型的驱动
driver = gdal.GetDriverByName('HFA')

# 将流指向新的地方
dst_ds = driver.CreateCopy(
    'example.img', src_ds)
```

栅格投影重置方式一：gdal.Warp()

与矢量重新投射修改形变的方式不同，栅格子区域还需要考虑到形变后每个波段上产生的新像素怎么去取舍数值和裁剪的重采样（ReSample）。

使用系统库绑定的自动化工具解决是第一要义。

```
root_ds = gdal.Open('MOD09A1.xxxxx.hdf')

# 对于 HDF 这个数据类型比较特殊，内含嵌套的多个子类数据组
ds_list = root_ds.GetSubDatasets()

# 调用 Warp API ，直接对第一子系统传入参数和我们想落地的EPSG等角横轴墨卡托投影中。
gdal.Warp('reprojection.tif', ds_list[0][0], dstSRS='EPSG:32649')
del root_ds
```

栅格投影重置方式二：底层 API 重构坐标

西北大学

如果要完全深入去使用函数操作：则需要非常深厚的物理数学知识对 Geotransform (六参数方程) 和边界的形变情况进行估算。

```python
def reproject(src_file, dst_file, epsg_to):
    # ...  (获取原参数与目标坐标)
    tx = osr.CoordinateTransformation(src_srs, dst_srs)

    # 我们需要手工按照四角点投影变化后，预估出重投影后新网格的面积最大极值
    (ulx, uly, _) = tx.TransformPoint(srs_trans[0], srs_trans[3])
    (urx, ury, _) = tx.TransformPoint(...) #右等点测算省略

    # 并按照新的比例尺去创建一张新的画布
    dst_ds = driver.Create(dst_file, (max_x - min_x) / 宽, (max_y - min_y) / 高, 1, 类型)
    dst_trans = (min_x, p_width, srs_trans[2], max_y, ...) #重构六参数

    dst_ds.SetGeoTransform(dst_trans)
    dst_ds.SetProjection(dst_srs.ExportToWkt())

    # 运用 ReprojectImage 按照双线性采样 GRA_Bilinear 投射转移波段信息
    gdal.ReprojectImage(src_ds, dst_ds, src_wkt, dst_wkt, gdal.GRA_Bilinear)
    dst_ds.FlushCache()
```

引言与简介

空间矢量数据

空间栅格数据

工具集与自动化处理

本章小结

GDAL 命令行工具

GDAL 除了库以外，也是一个功能异常齐全的后台可执行工具（包括 C 程序和基于 python 包装的衍生命令）。使用它可以不写一行代码对整个文件夹的各种数据完成格式转换。

在你的终端/PowerShell 通过声明带有工具标识的文件名称直接可激活这些独立引擎。如输入 `gdalinfo --help` 可查看各种各样的指令提示系统。

- 常用的基本控制标记包括诸如 `--version` 或者 `--formats` 了解系统当前状态。
- 使用 `-of` 指定目标导出格式如 `GTiff` 或 `ENVI`。
- 使用 `-ot` 控制数据类型的深浅变化（如 `UInt16` 或者 `Float32`）。
- 使用 `-t_srs` 来控制目标输出的 EPSG 坐标等。

实战常用命令速览 (1/2)

西北大学

gdalinfo

查询数据的最快利器，能够列出包含投影类型、像元尺寸、覆盖地理坐标范围及内部四至经纬及各种嵌套。

```
$ gdalinfo XiAn-AOI.tif
```

gdal_translate

擅长完成大小的调整、波段转换格式的改写。例如使用 `-tr` 去调整新的长宽尺寸，`-r bilinear` 提供双线性重采样法则。

```
$ gdal_translate -tr 100 100 \
  -r bilinear \
  a.tif new_a.tif
```

实战常用命令速览 (2/2)

gdalwarp 与投射

通过指定投影将该地的图转为符合区域要求的研究区展示模型。

```
$ gdalwarp -t_srs EPSG:4490 \
    origin.tif cgcs.tif
```

gdalbuildvrt 建立虚拟框架

它不进行物理剪切或叠加。而是提供一个包含了文件关系指向记录 *.VRT，极大的减轻了多步骤带来的空间和算力浪费，可以在分析的最后一刻真正输出文件。

Python 与系统进程的结合

基于 Python subprocess 子进程管理库，我们也可以抛开口口相传的脚本，将我们的系统指令通过代码批量灌入操作系统的引擎中（如把一百张图批量生成 VRT 等等）。

```python
import subprocess

# 使用 subprocess 调用控制台打印图像信息记录，同时并能接受捕捉其中的反馈字符串用以提取结果:
result = subprocess.run(
    ['gdalinfo', r"C:\Data\wsiearth.tif"],
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    check=True,
    universal_newlines=True
)

print(result.stdout)
```

结合 Pathlib 的扫描匹配功能实现文件夹内图像的联动合成并使用上述系统脚本是极度常用的套路选择。

引言与简介

空间矢量数据

空间栅格数据

工具集与自动化处理

本章小结

核心知识梳理

知识核心要点

- **熟悉基本体系**：通过 Python + Conda 的 GDAL 环境配置方式；了解底层矢量核心四件套 `Dataset`, `Layer`, `Feature`, `Geometry` 与 栅格组件。
- **读取与分析方式**：无论格式文件再特殊，都可以用 `Open()` 系函数配合多项提取获取源数据，针对栅格可以用过 numpy 等多维数组交互。
- **形变转换与构建**：对于转换都拥有内置包裹的自动化脚本执行或者进行底层的遍历坐标/像素矩阵重建机制。
- **实操工具熟练**：尝试熟练直接使用不写代码的控制台命令行交互（`gdal_translate`, `warp`）以及如何在代码中派生调用。

牛刀小试

1. 现有一气象站点数据点图层 (`storms.shp`)，其中包含字段 `TotalCount`（记录该站点的降雨次数）与 `TotalStorm`（暴雨降雨量），请思考如何编写逻辑，输出降雨次数在 10 次 以上的站点所有降雨量总计?
2. 给定土地利用栅格图（`landuse.tif`）和坡度栅格图（`slope.tif`），请尝试编写 Python 代码，统计土地利用图斑的坡度信息。